🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

นายปองภพ ศรีรักษ์ 673380279-7 sec.2

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

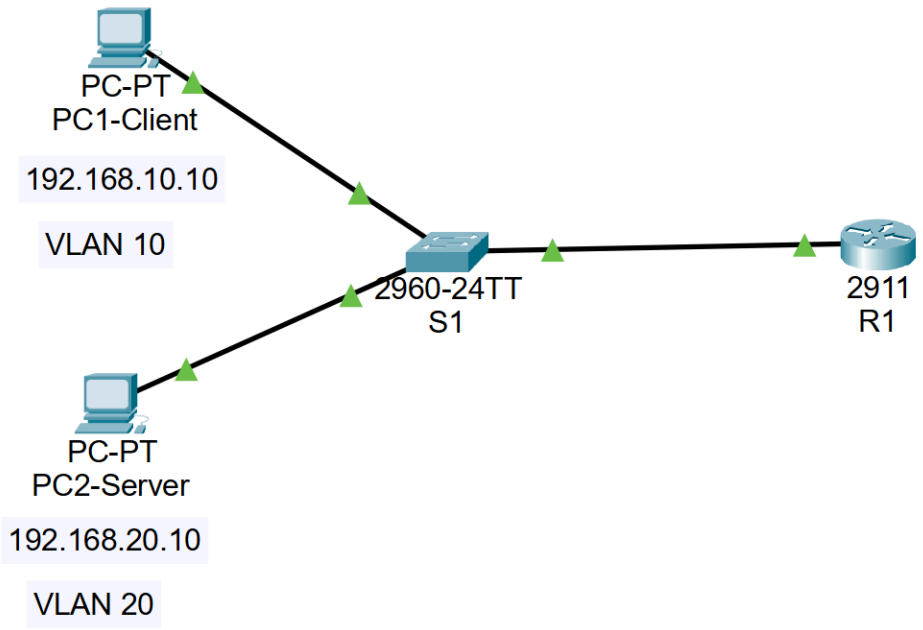
- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

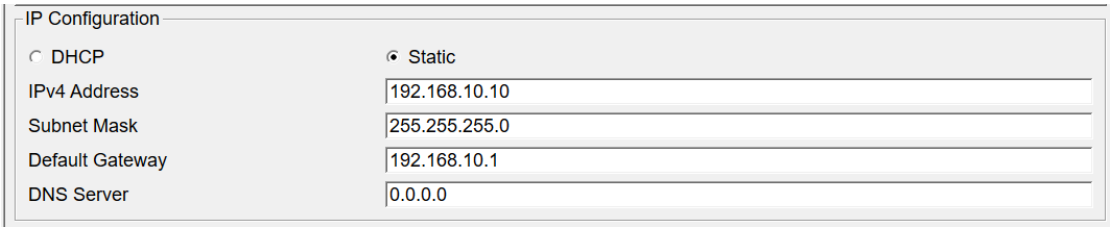
```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```



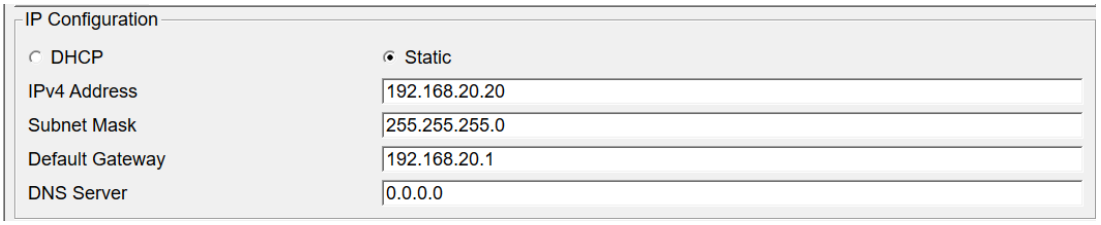
📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

PC1-Client



PC2-Server



🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
Router>
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#
R1(config)#interface g0/0
R1(config-if)#no shutdown
```

```
R1(config)#interface g0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#
```

```
R1(config-subif)#interface g0/0.20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#
```

```
R1#write memory
Building configuration...
[OK]
R1#
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

```
Switch>
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#
```

```
S1(config-vlan)#
S1(config-vlan)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

S1(config-if)#
S1(config-if)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#
S1(config-if)#
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#
S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#write memory
Building configuration...
[OK]
S1#
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time=1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  down                  down
GigabitEthernet0/2     unassigned      YES unset  down                  down
Vlan1                  unassigned      YES unset  up                    down
R1#
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
S1#show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    00e0.f706.4e01    DYNAMIC     Gig0/1
  10    00e0.f706.4e01    DYNAMIC     Gig0/1
  10    00e0.f72e.9544    DYNAMIC     Fa0/1
  20    000b.be8e.7647    DYNAMIC     Fa0/2
  20    00e0.f706.4e01    DYNAMIC     Gig0/1
S1#
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
PS D:\Comproject\Computer_network\automation\week01-mime-typing\server> python main_enhanced.py --host 127.0.0.1 --port 65432
--storage ../received_files --verbose
[20:44:33] INFO: Starting server with host=127.0.0.1, port=65432, storage=../received_files
[20:44:33] INFO: Created storage directory: ../received_files
[20:44:33] INFO: Server listening on 127.0.0.1:65432
[20:47:16] INFO: Connection from ('127.0.0.1', 53497)
[20:47:16] INFO: Receiving text/plain (36 bytes)
[20:47:16] INFO: Saved to ../received_files\received_84b1.plain
[20:47:16] WARNING: Connection error with ('127.0.0.1', 53497): [WinError 10053] An established connection was aborted by the
software in your host machine
[20:47:16] INFO: Connection with ('127.0.0.1', 53497) closed
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

```
PS D:\Comproject\Computer_network\automation\week01-mime-typing\client> python main_enhanced.py ../assets/notes.txt ../assets/
sample.png --host 127.0.0.1 --port 65432 --timeout 10 --verbose
[20:47:16] INFO: Starting client with 2 files to 127.0.0.1:65432
[20:47:16] INFO: Connected to 127.0.0.1:65432
[20:47:16] INFO: Sent ../assets/notes.txt (36 bytes) as text/plain
[20:47:16] INFO: Sent ../assets/sample.png (108 bytes) as image/png
[20:47:16] INFO: Connection closed. Sent: 2, Failed: 0
[20:47:16] INFO: All files sent successfully
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

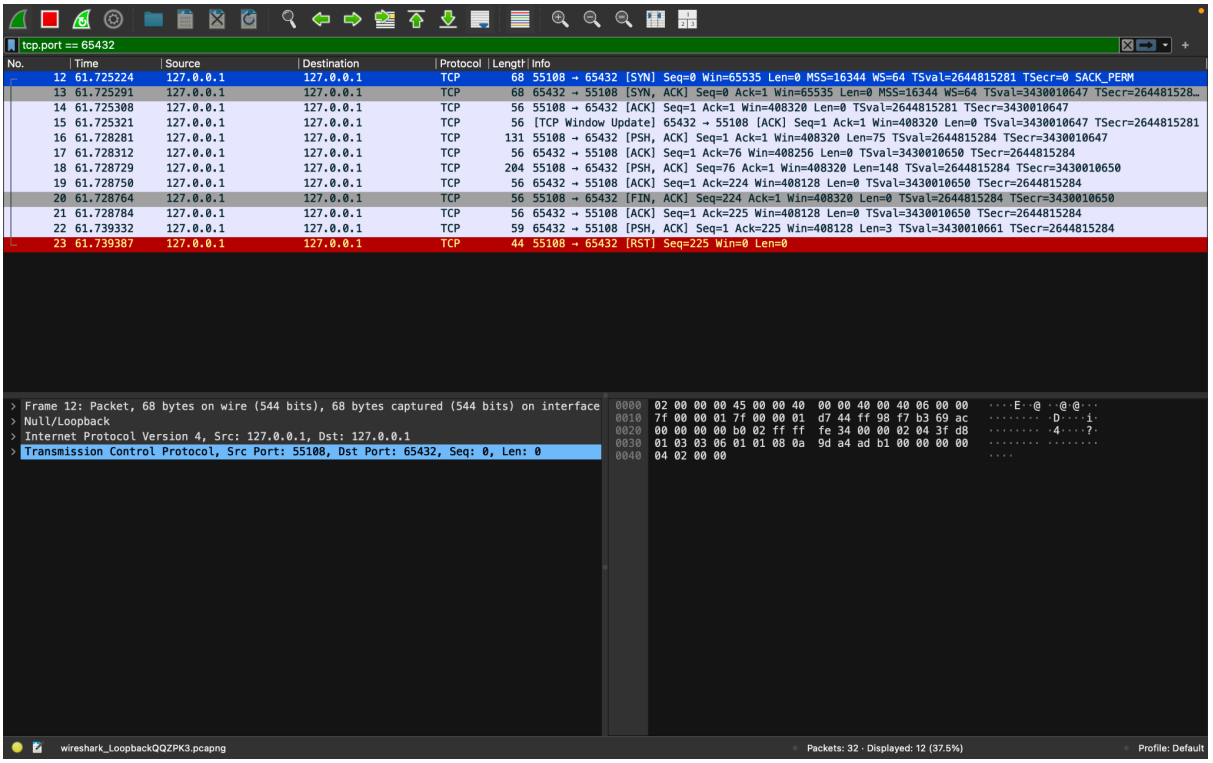
Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.



Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

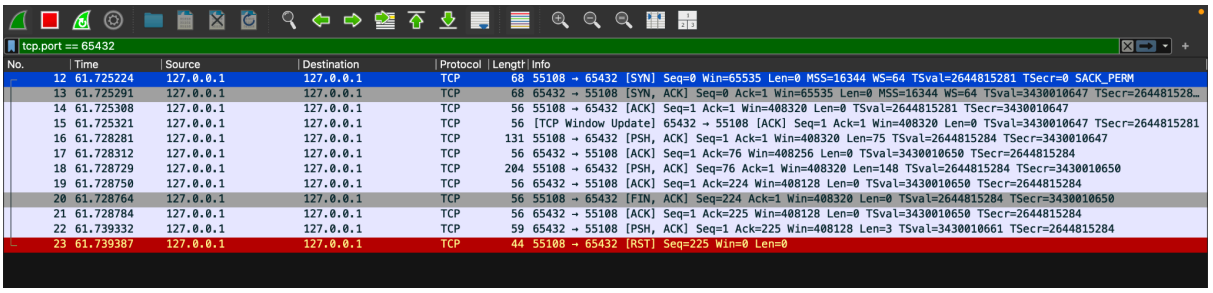
Confirm MAC resolution.

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.



tcp.port == 65432

No.	Time	Source	Destination	Protocol	Length	Info
12	61.725224	127.0.0.1	127.0.0.1	TCP	68	55108 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2644815281 TSecr=0 SACK_PERM
13	61.725291	127.0.0.1	127.0.0.1	TCP	68	65432 → 55108 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3430010647 TSecr=264481528…
14	61.725308	127.0.0.1	127.0.0.1	TCP	56	55108 → 65432 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2644815281 TSecr=3430010647
15	61.725321	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 65432 → 55108 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3430010647 TSecr=2644815281
16	61.728281	127.0.0.1	127.0.0.1	TCP	131	55108 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=75 TSval=2644815284 TSecr=3430010647
17	61.728312	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=76 Win=408256 Len=0 TSval=3430010650 TSecr=2644815284
18	61.728729	127.0.0.1	127.0.0.1	TCP	204	55108 → 65432 [PSH, ACK] Seq=76 Ack=1 Win=408320 Len=148 TSval=2644815284 TSecr=3430010650
19	61.728750	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=224 Win=408128 Len=0 TSval=3430010650 TSecr=2644815284
20	61.728764	127.0.0.1	127.0.0.1	TCP	56	55108 → 65432 [FIN, ACK] Seq=224 Ack=1 Win=408320 Len=0 TSval=2644815284 TSecr=3430010650
21	61.728784	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=225 Win=408128 Len=0 TSval=3430010650 TSecr=2644815284
22	61.739332	127.0.0.1	127.0.0.1	TCP	59	65432 → 55108 [PSH, ACK] Seq=1 Ack=225 Win=408128 Len=3 TSval=3430010661 TSecr=2644815284
23	61.739387	127.0.0.1	127.0.0.1	TCP	44	55108 → 65432 [RST] Seq=225 Win=0 Len=0

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.

```
{"mime_type": "text/plain", "size": 35}
Test content for final validation.
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
OK
```

D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

📝Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
   Ans TCP ส่งข้อมูลซ้ำเมื่อข้อมูลสูญหาย เพื่อให้การสื่อสารมีความน่าเชื่อถือ ข้อมูลต้องไปถึงครบและถูกต้อง แม้เครือข่ายจะมีความผิดพลาด

2. How does VLAN tagging affect broadcast domains?
   Ans การติดแท็ก VLAN ทำให้เครือข่ายถูกแบ่งเป็นหลาย broadcast domain อุปกรณ์คนละ VLAN จะไม่เห็น broadcast ของกันและกัน ต้องผ่านเราเตอร์จึงจะสื่อสารข้าม VLAN ได้

3. What role does MIME play compared to TCP headers?
   Ans การตัดสินใจเส้นทางเกิดขึ้นที่เราเตอร์ โดยดูจาก IP ปลายทางและตารางเส้นทาง

4. Where does routing decision occur?
   Ans MIME ทำหน้าที่บอก “ความหมายของข้อมูล” ส่วน TCP header ทำหน้าที่ดูแลการขนส่งให้ข้อมูลถึงปลายทางอย่างถูกต้อง

5. Why does ARP precede TCP communication?
   Ans ต้องใช้ ARP หา MAC address ก่อน ถึงจะส่งแพ็กเก็ต TCP ผ่านเครือข่ายท้องถิ่นได้

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy
- Reliability engineering
- Metadata semantics
- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.